

第 10 章 MSI 和 MSI-X 中断机制

在 PCI 总线中，所有需要提交中断请求的设备，必须能够通过 INTx 引脚提交中断请求，而 MSI 机制是一个可选机制。而在 PCIe 总线中，PCIe 设备必须支持 MSI 或者 MSI-X 中断请求机制，而可以不支持 INTx 中断消息。

在 PCIe 总线中，MSI 和 MSI-X 中断机制使用存储器写请求 TLP 向处理器提交中断请求，下文为简便起见将传递 MSI/MSI-X 中断消息的存储器写报文简称为 MSI/MSI-X 报文。不同的处理器使用了不同的机制处理这些 MSI/MSI-X 中断请求，如 PowerPC 处理器使用 MPIC 中断控制器处理 MSI/MSI-X 中断请求，在第 10.2 节中将介绍这种处理情况；而 x86 处理器使用 FSB Interrupt Message 方式处理 MSI/MSI-X 中断请求。

不同的处理器对 PCIe 设备发出的 MSI 报文的解释并不相同。但是 PCIe 设备在提交 MSI 中断请求时，都是向 MSI/MSI-X Capability 结构中的 Message Address 的地址写 Message Data 数据，从而组成一个存储器写 TLP，向处理器提交中断请求。

有些 PCIe 设备还可以支持 Legacy 中断方式^①。但是 PCIe 总线并不鼓励其设备使用 Legacy 中断方式，在绝大多数情况下，PCIe 设备使用 MSI 或者 MSI/X 方式进行中断请求。

PCIe 总线提供 Legacy 中断方式的主要原因是，在 PCIe 体系结构中，存在许多 PCI 设备，而这些设备通过 PCIe 桥连接到 PCIe 总线中。这些 PCI 设备可能并不支持 MSI/MSI-X 中断机制，因此必须使用 INTx 信号进行中断请求。

当 PCIe 桥收到 PCI 设备的 INTx 信号后，并不能将其直接转换为 MSI/MSI-X 中断报文，因为 PCI 设备使用 INTx 信号进行中断请求的机制与电平触发方式类似，而 MSI/MSI-X 中断机制与边沿触发方式类似。这两种中断触发方式不能直接进行转换。因此当 PCI 设备的 INTx 信号有效时，PCIe 桥将该信号转换为 Assert_INTx 报文，当这些 INTx 信号无效时，PCIe 桥将该信号转换为 Deassert_INTx 报文。

与 Legacy 中断方式相比，PCIe 设备使用 MSI 或者 MSI-X 中断机制，可以消除 INTx 这个边带信号，而且可以更加合理地处理 PCIe 总线的“序”。目前绝大多数 PCIe 设备使用 MSI 或者 MSI-X 中断机制提交中断请求。

MSI 和 MSI-X 机制的基本原理相同，其中 MSI 中断机制最多只能支持 32 个中断请求，而且要求中断向量连续，而 MSI-X 中断机制可以支持更多的中断请求，而并不要求中断向量连续。与 MSI 中断机制相比，MSI-X 中断机制更为合理。本章将首先介绍 MSI/MSI-X Capability 结构，之后分别以 PowerPC 处理器和 x86 处理器为例介绍 MSI 和 MSI-X 中断机制。

10.1 MSI/MSI-X Capability 结构

PCIe 设备可以使用 MSI 或者 MSI-X 报文向处理器提交中断请求，但是对于某个具体的

① 通过发送 Assert_INTx 和 Deassert_INTx 消息报文进行中断请求，即虚拟中断线方式。

PCIe 设备，可能仅支持一种方式。在 PCIe 设备中含有两个 Capability 结构，一个是 MSI Capability 结构，另一个是 MSI-X Capability 结构。通常情况下一个 PCIe 设备仅包含一种结构，或者为 MSI Capability 结构，或者为 MSI-X Capability 结构。

10.1.1 MSI Capability 结构

MSI Capability 结构共有四种组成方式，分别是 32 位和 64 位的 Message 结构，32 位和 64 位带中断 Masking 的结构。MSI 报文可以使用 32 位地址或者 64 位地址，而且可以使用 Masking 机制使能或者禁止某个中断源。MSI Capability 寄存器的结构如图 10-1 所示。

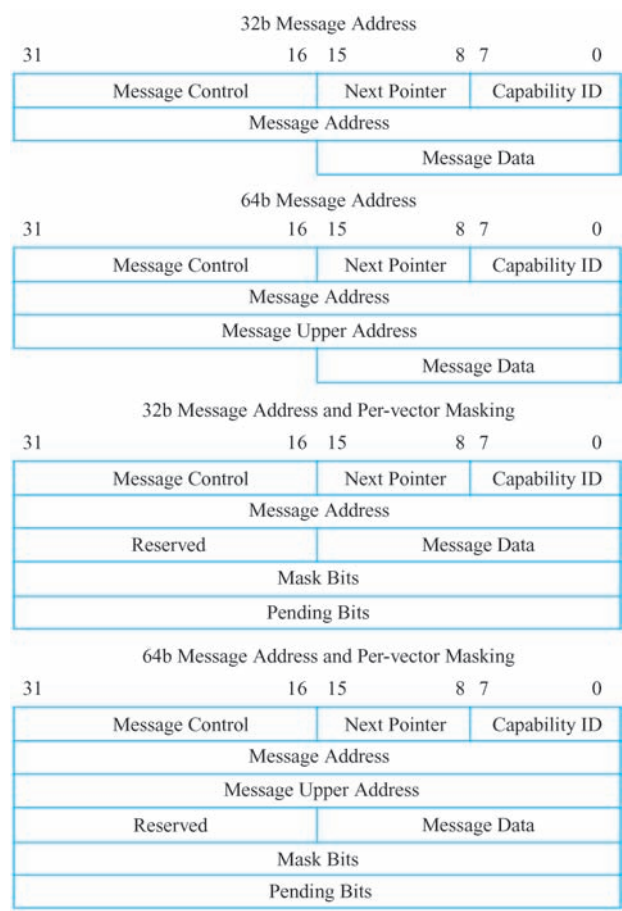


图 10-1 MSI Capability 寄存器结构

- Capability ID 字段记载 MSI Capability 结构的 ID 号，其值为 0x05。在 PCIe 设备中，每一个 Capability 结构都有唯一的 ID 号。
- Next Pointer 字段存放下一个 Capability 结构的地址。
- Message Control 字段。该字段存放当前 PCIe 设备使用 MSI 机制进行中断请求的状态与控制信息，如表 10-1 所示。

表 10-1 MSI Cabability 结构的 Message Control 字段

Bits	定 义	描 述
15 : 9	Reserved	保留位。系统软件读取该字段时将返回全零，对此字段写无意义
8	Per-vector Masking Capable	该位为 1 时，表示支持带中断 Masking 的结构；如果为 0，表示不支持带中断 Masking 的结构。该位对系统软件只读，在 PCIe 设备初始化时设置
7	64 bit Address Capable	该位为 1 时，表示支持 64 位地址结构；如果为 0，表示只能支持带 32 位地址结构。该位对系统软件只读，在 PCIe 设备初始化时设置
6 : 4	Multiple Message Enable	该字段可读写，表示软件分配给当前 PCIe 设备的中断向量数目。系统软件根据 Multiple Message Capable 字段的大小确定该字段的值。在系统的中断向量资源并不紧张时，Multiple Message Capable 字段和该字段的值相等；而资源紧张时，该字段的值可能小于 Multiple Message Capable 字段的值
3 : 1	Multiple Message Capable	该字段对系统软件只读，表示当前 PCIe 设备可以使用几个中断向量号，在不同的 PCIe 设备中该字段的值不同。当该字段为 0b000 时，表示 PCIe 设备可以使用 1 个中断向量；为 0b001、0b010、0b011、0b100 和 0b101 时，表示使用 4、8、16 和 32 个中断向量；而 0b110 和 0b111 为保留位。该字段与 Multiple Message Enable 字段的含义不同，该字段表示当前 PCIe 设备支持的中断向量个数，而 Multiple Message Enable 字段是系统软件分配给 PCIe 设备实际使用的中断向量个数
0	MSI Enable	该位可读写，是 MSI 中断机制的使能位。该位为 1 而且 MSI-X Enable 位为 0 时，当前 PCIe 设备可以使用 MSI 中断机制，此时 Legacy 中断机制被禁止。一个 PCIe 设备的 MSI Enable 和 MSI-X Enable 位都被禁止时，将使用 INTx 中断消息报文发出/结束中断请求 [⊖]

- Message Address 字段。当 MSI Enable 位有效时，该字段存放 MSI 存储器写事务的目的地址的低 32 位。该字段的 31:2 字段有效，系统软件可以对该字段进行读写操作；该字段的第 1~0 位为 0。
- Message Upper Address 字段。如果 64 bit Address Capable 位有效，该字段存放 MSI 存储器写事务的目的地址的高 32 位。
- Message Data 字段，该字段可读写。当 MSI Enable 位有效时，该字段存放 MSI 报文使用的数据。该字段保存的数值与处理器系统相关，在 PCIe 设备进行初始化时，处理器将初始化该字段，而且不同的处理器填写该字段的规则并不相同。如果 Multiple Message Enable 字段不为 0b000 时（即该设备支持多个中断请求时），PCIe 设备可以通过改变 Message Data 字段的低位数据发送不同的中断请求。
- Mask Bits 字段。PCIe 总线规定当一个设备使用 MSI 中断机制时，最多可以使用 32 个中断向量，从而一个设备最多可以发送 32 种中断请求。Mask Bits 字段由 32 位组成，其中每一位对应一种中断请求。当相应位为 1 时表示对应的中断请求被屏蔽，为 0 时表示允许该中断请求。系统软件可读写该字段，系统初始化时该字段为全 0，表示允许所有中断请求。该字段和 Pending Bits 字段对于 MSI 中断机制是可选字段，但是 PCIe 总线规范强烈建议所有 PCIe 设备支持这两个字段。
- Pending Bits 字段。该字段对于系统软件是只读位，PCIe 设备内部逻辑可以改变该字段的值。该字段由 32 位组成，并与 PCIe 设备使用的 MSI 中断一一对应。该字段需要

⊖ 此时 PCI 设备配置空间 Command 寄存器的“Interrupt Disable”位为 1。

与 Mask Bits 字段联合使用。

当 Mask Bits 字段的相应位为 1 时, 如果 PCIe 设备需要发送对应的中断请求, Pending Bits 字段的对应位将被 PCIe 设备的内部逻辑置 1, 此时 PCIe 设备并不会使用 MSI 报文向中断控制器提交中断请求; 当系统软件将 Mask Bits 字段的相应位从 1 改写为 0 时, PCIe 设备将发送 MSI 报文向处理器提交中断请求, 同时将 Pending Bit 字段的对应位清零。在设备驱动程序的开发中, 有时需要联合使用 Mask Bits 和 Pending Bits 字段防止处理器丢弃中断请求^①。

10.1.2 MSI-X Capability 结构

MSI-X Capability 中断机制与 MSI Capability 的中断机制类似。PCIe 总线引出 MSI-X 机制的主要目的是为了扩展 PCIe 设备使用中断向量的个数, 同时解决 MSI 中断机制要求使用中断向量号连续所带来的问题。

MSI 中断机制最多只能使用 32 个中断向量, 而 MSI-X 可以使用更多的中断向量。目前 Intel 的许多 PCIe 设备支持 MSI-X 中断机制。与 MSI 中断机制相比, MSI-X 机制更为合理。首先 MSI-X 可以支持更多的中断请求, 但这并不是引入 MSI-X 中断机制最重要的原因。因为对于多数 PCIe 设备, 32 种中断请求已经足够了。而引入 MSI-X 中断机制的主要原因是, 使用该机制不需要中断控制器分配给该设备的中断向量号连续。

如果一个 PCIe 设备需要使用 8 个中断请求且使用 MSI 机制时, Message Data 的[2:0]字段可以为 0b000~0b111, 因此可以发送 8 个中断请求, 但是这 8 个中断请求的 Message Data 字段必须连续。在许多中断控制器中, Message Data 字段连续也意味着中断控制器需要为这个 PCIe 设备分配 8 个连续的中断向量号。

有时在一个中断控制器中, 虽然具有 8 个以上的中断向量号, 但是很难保证这些中断向量号是连续的。因此中断控制器将无法为这些 PCIe 设备分配足够的中断请求, 此时该设备的“Multiple Message Enable”字段将小于“Multiple Message Capable”。

而使用 MSI-X 机制可以合理解决该问题。在 MSI-X Capability 结构中, 每一个中断请求都使用独立的 Message Address 字段和 Message Data 字段, 从而中断控制器可以更加合理地为该设备分配中断资源。

与 MSI Capability 寄存器相比, MSI-X Capability 寄存器使用一个数组存放 Message Address 字段和 Message Data 字段, 而不是将这两个字段放入 Capability 寄存器中, 本书将这个数组称为 MSI-X Table。从而当 PCIe 设备使用 MSI-X 机制时, 每一个中断请求可以使用独立的 Message Address 字段和 Message Data 字段。

除此之外 MSI-X 中断机制还使用了独立的 Pending Table 表, 该表用来存放与每一个中断向量对应的 Pending 位。这个 Pending 位的定义与 MSI Capability 寄存器的 Pending 位类似。MSI-X Table 和 Pending Table 存放在 PCIe 设备的 BAR 空间中。MSI-X 机制必须支持这个 Pending Table, 而 MSI 机制的 Pending Bits 字段是可选的。

1. MSI-X Capability 结构

MSI-X Capability 结构比 MSI Capability 结构复杂一些。在该结构中, 使用 MSI-X Table 存

① MSI 机制提交中断请求的方式类似于边界触发方式, 而使用边界触发方式时, 处理器可能会丢失某些中断请求, 因此在设备驱动程序的开发过程中, 可能需要使用这两个字段。

放该设备使用的所有 Message Address 和 Message Data 字段，这个表格存放在该设备的 BAR 空间中，从而 PCIe 设备可以使用 MSI-X 机制时，中断向量号可以不连续，也可以申请更多的中断向量号。MSI-X Capability 结构的组成方式如图 10-2 所示。

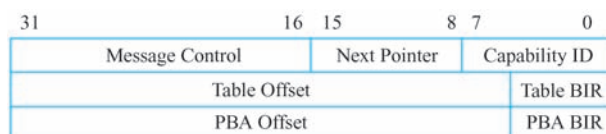


图 10-2 MSI-X Capability 结构的组成方式

上图中各字段的含义如下所示。

- Capability ID 字段记载 MSI-X Capability 结构的 ID 号，其值为 0x11。在 PCIe 设备中，每个 Capability 都有唯一的 ID 号。
- Next Pointer 字段存放下一个 Capability 结构的地址。
- Message Control 字段，该字段存放当前 PCIe 设备使用 MSI-X 机制进行中断请求的状态与控制信息，如表 10-2 所示。

表 10-2 MSI-X Capability 结构的 Message Control 字段

Bits	定 义	描 述
15	MSI-X Enable	该位可读写，是 MSI-X 中断机制的使能位，复位值为 0，表示不使能 MSI-X 中断机制。该位为 1 且 MSI Enable 位为 0 时，当前 PCIe 设备使用 MSI-X 中断机制，此时 INTx 和 MSI 中断机制被禁止。当 PCIe 设备的 MSI Enable 和 MSI-X Enable 位为 0 时，将使用 INTx 中断消息报文发出/结束中断请求
14	Function Mask	该位可读写，是中断请求的全局 Mask 位，复位值为 0。如果该位为 1，该设备所有的中断请求都将被屏蔽；如果该位为 0，则由 Per Vector Mask 位决定是否屏蔽相应的中断请求。Per Vector Mask 位在 MSI-X Table 中定义，详见下文
10 : 0	Table Size	MSI-X 中断机制使用 MSI-X Table 存放 Message Address 字段和 Message Data 字段。该字段用来存放 MSI-X Table 的大小，该字段对系统软件只读

- Table BIR (BAR Indicator Register)。该字段存放 MSI-X Table 所在的位置，PCIe 总线规范规定 MSI-X Table 存放在设备的 BAR 空间中。该字段表示设备使用 BAR0~5 寄存器中的哪个空间存放 MSI-X Table。该字段由三位组成，其中 0b000~0b101 与 BAR0~5 空间一一对应。
- Table Offset 字段。该字段存放 MSI-X Table 在相应 BAR 空间中的偏移。
- PBA (Pending Bit Array) BIR 字段。该字段表示 Pending Table 存放在 PCIe 设备的哪个 BAR 空间中，0 表示 BAR0 空间，1 表示 BAR1 空间，依此类推。在通常情况下，Pending Table 和 MSI-X Table 存放在 PCIe 设备的同一个 BAR 空间中。
- PBA Offset 字段。该字段存放 Pending Table 在相应 BAR 空间中的偏移。

2. MSI-X Table

MSI-X Table 的组成结构如图 10-3 所示。

由该图可见，MSI-X Table 由多个 Entry 组成，其中每个 Entry 与一个中断请求对应。每个 Entry 中有四个参数，其含义如下所示。

DWORD 3	DWORD 2	DWORD 1	DWORD 0	
Vector Control	Msg Data	Msg Upper Addr	Msg Addr	Entry 0
Vector Control	Msg Data	Msg Upper Addr	Msg Addr	Entry 1
Vector Control	Msg Data	Msg Upper Addr	Msg Addr	Entry 2
...	...	...	...	...
Vector Control	Msg Data	Msg Upper Addr	Msg Addr	Entry N-1

图 10-3 MSI-X Table 的组成结构

- Msg Addr。当 MSI-X Enable 位有效时，该字段存放 MSI-X 存储器写事务的目的地址的低 32 位。该双字的 31:2 字段有效，系统软件可读写；1:0 字段复位时为 0，PCIe 设备可以根据需要将这个字段设为只读，或者可读写。不同的处理器填入该寄存器的数据并不相同。
- Msg Upper Addr，该字段可读写，存放 MSI-X 存储器写事务的目的地址的高 32 位。
- Msg Data，该字段可读写，存放 MSI-X 报文使用的数据。其定义与处理器系统使用的中断控制器和 PCIe 设备相关。
- Vector Control，该字段可读写。该字段只有第 0 位（即 Per Vector Mask 位）有效，其他位保留。当该位为 1 时，PCIe 设备不能使用该 Entry 提交中断请求；为 0 时可以提交中断请求。该位在复位时为 0。Per Vector Mask 位的使用方法与 MSI 机制的 Mask 位类似。

3. Pending Table

Pending Table 的组成结构如图 10-4 所示。

63	0	
Pending Bits 0 Through 63		QWORD 0
Pending Bits 64 Through 127		QWORD 1
...		...
Pending Bits(N-1 div 64)×64 Through N-1		QWORD((N-1) div 64)

图 10-4 Pending Table 的组成结构

如上图所示，在 Pending Table 中，一个 Entry 由 64 位组成，其中每一位与 MSI-X Table 中的一个 Entry 对应，即 Pending Table 中的每一个 Entry 与 MSI-X Table 的 64 个 Entry 对应。与 MSI 机制类似，Pending 位需要与 Per Vector Mask 位配置使用。

当 Per Vector Mask 位为 1 时，PCIe 设备不能立即发送 MSI-X 中断请求，而是将对应的 Pending 位置 1；当系统软件将 Per Vector Mask 位清零时，PCIe 设备需要提交 MSI-X 中断请求，同时将 Pending 位清零。

10.2 PowerPC 处理器如何处理 MSI 中断请求

PowerPC 处理器使用 OpenPIC 中断控制器或者 MPIC 中断控制器，处理外部中断请求。其中 MPIC 中断控制器基于 OpenPIC 中断控制器，但是做出了许多增强，目前 Freescale 新推出的 PowerPC 处理器，其中断控制器多与 MPIC 兼容。

值得注意的是，PowerPC 处理器和 x86 处理器处理 MSI 报文的方式有较大的不同。其中

x86 处理器使用的机制比 PowerPC 处理器更为合理，但是 PowerPC 处理器使用的方法使用的硬件资源相对较少。本节将 MPC8572 处理器为例说明 MSI 机制的处理过程，在第 10.3 节介绍 x86 处理器如何实现 MSI 机制。

MPIC 中断控制器是 Freescale 的 PowerPC 处理器使用的通用中断控制器，目前基于 E500 内核的处理器，如 MPC854x、8572 等处理器使用这种中断控制器。目前 Freescale 使用 QorIQ 架构，该架构使用的中断控制器与 MPIC 兼容。

使用 MPIC 中断控制器处理 MSI 中断时，PCIe 设备的 MSI 报文，其目的地址为 MPIC 中断控制器的 MSIIR 寄存器。当该寄存器被 PCIe 设备写入后，MPIC 中断控制器将向处理器内核提交中断请求，之后处理器再通过读取 MPIC 中断控制器的 ACK 寄存器获得中断向量号，并进行相应的中断处理。这种方式与 x86 处理器的 FSB Interrupt Message 机制相比，处理器需要读取 ACK 寄存器，从而中断处理的延时较大。

目前 Freescale 的 P4080 处理器对 MPIC 中断控制器进行了优化。在 P4080 处理器中，MPIC 中断控制器向处理器提交中断请求的同时，也向处理器内核提交中断向量，处理器内核不必读取 ACK 寄存器获得中断向量，从而缩短了中断处理延时。使用这种方法的效率与 x86 处理器使用的 FSB Interrupt Message 机制相当。

目前 Freescale 并没有完全公开 P4080 处理器的实现细节，因此本节仍以 MPC8572 处理器为例介绍 PCIe 设备的 MSI 中断请求。在 MPC8572 处理器中，MPIC 中断控制器的拓扑结构如图 10-5 所示。

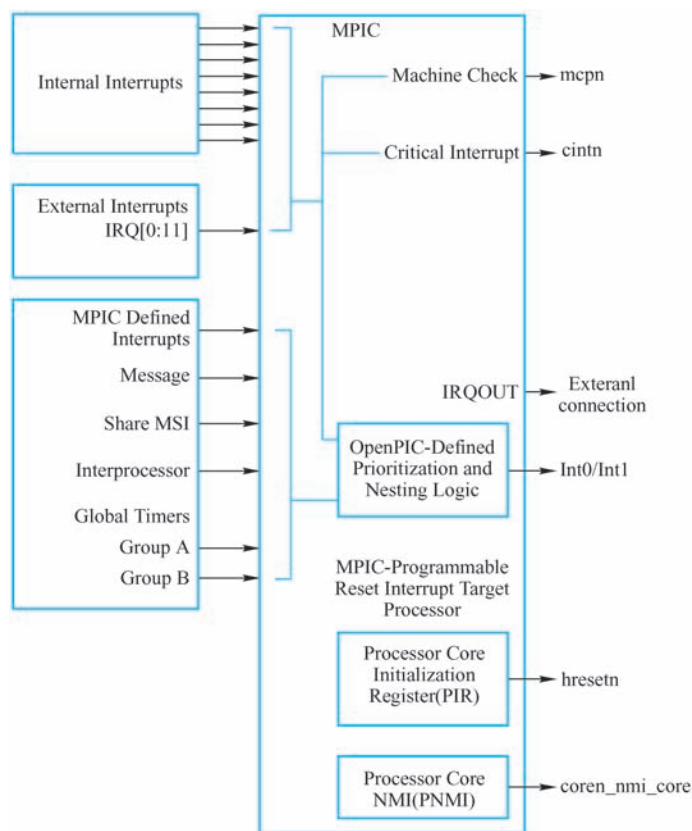


图 10-5 MPIC 中断控制器的拓扑结构

由上图所示，MPIC 中断控制器可以处理内部中断请求^①、外部中断请求，Message、处理器间中断请求和 Share MSI 中断请求等。而 MPIC 中断控制器使用 Int0、Int1 等中断线向处理器提交这些中断请求。其中 Internal Interrupts 和 External Interrupts 模块处理 MPC8572 内部和外部的中断请求，而 Share MSI 处理来自 PCIe 设备的 MSI 或者 MSI-X 中断请求。

当 MPIC 中断控制器收到 MSI 报文后，将使用中断线 Int0、Int1 或者 cintn 向处理器内核提交中断请求。处理器内核被中断后，将读取 ACK 寄存器获得中断向量，然后执行相应的中断服务例程。为此 PowerPC 处理器设置了一系列寄存器，如下文所示。

10.2.1 MSI 中断机制使用的寄存器

PowerPC 处理器设置了一系列寄存器，处理来自 PCIe 设备的 MSI 报文，其中最重要的寄存器是 MSIIR 寄存器。在 PowerPC 处理器系统中，PCIe 设备 Message Address 寄存器中存放的值都为 MSIIR 寄存器的物理地址，而 Message Data 寄存器中存放的数据也与 MSIIR 寄存器相关。

在 PowerPC 处理器系统中，MSI 机制的实现过程是 PCIe 设备向 MSIIR 寄存器写入指定的数据。MPIC 中断控制器发现该寄存器被写入后，将向处理器提交中断请求。处理器收到这个中断请求后，将通过读取 MPIC 中断控制器的 ACK 寄存器确定中断向量，并依此确定中断源。为此 PowerPC 处理器还设置了其他寄存器实现 MSI 中断机制。

1. MSIIR 寄存器

在 PowerPC 处理器中，MSIIR（Shared Message Signaled Interrupt Index Register）寄存器是实现 MSI 机制的重要寄存器。

当 PCIe 设备对 MSIIR 寄存器进行写操作时，MPC8572 处理器将使能 MSIR0~MSIR7 寄存器的相应位，从而向 MPIC 中断控制器提交中断请求，而中断控制器将转发这个中断请求，由处理器进一步处理。该寄存器各字段的详细描述如表 10-3 所示。

表 10-3 MSIIR 寄存器

Bits	定 义	描 述
27~31	IBS	该字段用来选择 MSIR0~MSIR7 寄存器的对应位。0b00000 对应 SH0；0b00001 对应 SH1；0b00010 对应 SH2；以此类推 0b11111 对应 SH31
24~26	SRS	该字段用来选择 MSIR0~MSIR7 寄存器。0b000 对应 MSIR0；0b001 对应 MSIR1；0b010 对应 MSIR2；以此类推 0b111 对应 MSIR7
0~24		保留

PCIe 设备通过 MSI 机制，向此寄存器写入数据时，MSIR0~MSIR7 寄存器的相应位 SH0~SH31 将有一位置 1。例如 PCIe 设备向 MSIIR 寄存器写入 0xFF00000 时，MSIR7 寄存器的 SH31 位将置 1（SRS 字段为 0b111 用来选择 MSIR7，而 IBS 字段为 0b11111 用来选择 SH31）。

2. MSIR 寄存器组

MSIR（Shared Message Signaled Interrupt Registers）寄存器组共由 8 个寄存器组成，分别

① PowerPC 处理器中含有许多模块，如千兆以太网、ATM 等，这些模块包含在芯片内部，由这些内部模块发起的中断请求，被称为内部中断请求。

为 MSIR0~MSIR7。其中每一个 MSIRx 寄存器中有 32 个有效位，分别为 SH0~SH31。当 PCIe 设备对 MSIIR 寄存器进行写操作时，某一个 MSIIRx 寄存器的某个 SH 位将被置为有效。系统软件通过读取该寄存器获得中断源，该寄存器读清除，对此寄存器进行写操作没有意义。

该寄存器组的大小决定了一个 PowerPC 处理器支持的 MSI 中断请求的个数。在 MPC8572 处理器中，有 8 个 MSIRx 寄存器，每个寄存器由 32 个有效位组成，因此 MPC8572 处理器最多能够处理 256 个 MSI 中断请求。该寄存器的结构如图 10-6 所示。

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
SH31	SH30	SH29	SH28	SH27	SH26	SH25	SH24	SH23	SH22	SH21	SH20	SH19	SH18	SH17	SH16
16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
SH15	SH14	SH13	SH12	SH11	SH10	SH9	SH8	SH7	SH6	SH5	SH4	SH3	SH2	SH1	SH0

图 10-6 MSIRx 寄存器的结构

3. MSISR 寄存器

MSISR 寄存器（Shared Message Signaled Interrupt Status Register）共由 8 个有效位组成，每一位对应一个 MSIR 寄存器。MPC8572 处理器设置该寄存器的主要目的是方便系统软件定位究竟是哪个 MSIR 寄存器中存在有效的中断请求。首先系统软件通过 MSISR 寄存器判断是哪个 MSIRx 寄存器存在有效请求，之后读取相应的 MSIRx 寄存器，该寄存器各字段的详细描述如表 10-4 所示。

表 10-4 MSISR 寄存器

Bits	定 义	描 述
0~23		保留
24~31	Sn	该字段由 8 位组成，每一位与一个 MSIR0~7 寄存器对应。该位为 0 时表示在 MSIRn 寄存器中没有有效位，即没有中断请求；该位为 1 时表示 MSIRn 寄存器中至少有一个有效位，即存在中断请求。Sn 位是 MSIRn 寄存器各个位的“与”，当 MSIRn 寄存器的相应位清除时，Sn 也将被清除

4. MSIVPR 寄存器组

MSIVPR（Shared Message Signaled Interrupt Vector/Priority Register）寄存器组由 8 个寄存器组成，分别为 MSIVPR0~7 寄存器。该组寄存器设置对应中断请求的优先级别和中断向量。其中每个 MSIVPR 寄存器对应一个 MSIR 寄存器，MSIVPR 寄存器各字段的详细解释如表 10-5 所示。

表 10-5 MSIVPR 寄存器

Bits	定 义	描 述
0	MSK	该位为 0，且 MSIR 寄存器的对应位为 1 时，则将向中断控制器提交中断请求；如果为 1 屏蔽该中断请求
1	A	该位为 0 时，表示 MPIC 中断控制器没有处理该中断请求；该位为 1 时，表示 MPIC 中断控制器正在处理该中断请求，或者该中断控制器准备处理该中断请求，这个中断请求将在 IPR（Interrupt Pending Regsiter）寄存器中排队等待处理，或者在 ISR（Interrupt Service Register）寄存器中正在被处理。该位的详细描述见 MPC8572 的数据手册
12~15	PRIORITY	OpenPIC 和 MPIC 中断控制器中为每一个中断请求设置了 0~15，共 16 个优先级。其中 1 的优先权最低，15 的优先权最高，0 表示禁止中断请求

(续)

Bits	定 义	描 述
16~31	VECTOR	该字段存放该中断的中断向量。当处理器读取 IACK 寄存器时, 将获得对应中断请求的中断向量

通过该组寄存器可以发现, 在 MPC8572 处理器系统中, PCIe 设备最多可以使用 8 个中断向量, 并可以共享这些中断向量。

5. MSIDR 寄存器组

MSIDR (Shared Message Signaled Interrupt Destination Registers) 寄存器组共由 8 个寄存器组成, 分别为 MSIDR0~7。其中每一个 MSIDRn 寄存器对应一个 MSIR 寄存器。

MPIC 中断控制器支持 Pass-through 方式, 在这种方式下, PowerPC 处理器可以使用外部中断控制器处理中断请求 (这种方法极少使用), 而不使用内部中断控制器。MPIC 中断控制器可以使用 cint# 和 int# 信号提交中断请求, 但是绝大多数系统软件都使用 int# 信号向处理器提交中断请求。

此外在 MPC8572 处理器中有两个 CPU, 分别为 CPU0 和 CPU1, MSI 机制提交的中断请求可以由 CPU0 或者 CPU1 处理。系统软件可以通过设置 MSIDRn 寄存器完成这些功能, 该寄存器各字段的详细描述如表 10-6 所示。

表 10-6 MSIDRn 寄存器

Bits	定 义	描 述
0	EP	为 1 时, 表示中断请求输出到 IRQ_OUT 由外部中断控制器处理; 为 0 时, 表示由 MPIC 中断控制器处理
1	CI0	为 1 时, 表示中断控制器使用 cint# 信号向 CPU0 提交中断请求
2	CI1	为 1 时, 表示中断控制器使用 cint# 信号向 CPU1 提交中断请求
30	P1	为 1 时, 表示中断控制器使用 int# 信号向 CPU0 提交中断请求
31	P0	为 1 时, 表示中断控制器使用 int# 信号向 CPU1 提交中断请求

10.2.2 系统软件如何初始化 PCIe 设备的 MSI Capability 结构

如果 PCIe 设备支持 MSI 机制, 系统软件首先设置该设备 MSI Capability 结构的 Message Address 和 Message Data 字段。如果该 PCIe 设备支持 64 位地址空间, 即 MSI Capability 寄存器的 64 bit Address Capable 位有效时, 系统软件还需要设置 Message Upper Address 字段。系统软件完成这些设置后, 将置 MSI Capability 结构的 MSI Enable 位有效, 使能该 PCIe 设备的 MSI 机制。

其中 Message Address 字段所填写的值是 MSIIR 寄存器在 PCI 总线域中的物理地址。在 PowerPC 处理器中, PCI 总线域与存储器域地址空间独立, 当 PCIe 设备访问存储器域的地址空间时, 需要通过 Inbound 寄存器组将 PCI 总线域地址空间转换为存储器域地址空间。

在 PowerPC 处理器中, PCIe 设备使用 MSI 机制访问 MSIIR 寄存器时, 可以不使用 Inbound 寄存器组进行 PCI 总线地址到处理器地址的转换。在 MPC8572 处理器中, 专门设置了一个 PEXCSRBAR 窗口[⊖], 进行 PCI 总线域到存储器域的地址转换, 使用这种方法可以节

⊖ 该窗口的大小为 1 MB, 其基地址由 PEXCSRBAR 寄存器确定。

省 Inbound 寄存器窗口，Linux PowerPC 使用了这种实现方式。

在 MPC8572 处理器中，MSIIR 寄存器的基地址为 CCSRBAR[Ⓐ]（Configuration, Control, and Status Base Address Register），其偏移为 0x1740。为支持 MSI 中断机制，系统软件需要使用 PEXCSRBAR 窗口将 MSIIR 寄存器映射到 PCI 总线域地址空间，即将 CCSRBAR 寄存器空间映射到 PCI 总线域地址空间。之后 PCIe 设备就可以通过 MSIIR 寄存器在 PCI 总线域的地址访问 MSIIR 寄存器。

Linux PowerPC 使用 setup_pci_pcsrbar 函数[Ⓑ]设置 PEXCSRBAR 窗口，该函数的源代码在 ./arch/powerpc/sysdev/fsl_pci.c 文件中，如源代码 10-1 所示，这段代码来自 Linux 2.6.30.5。

源代码 10-1 setup_pci_pcsrbar 函数

```
static void __init setup_pci_pcsrbar (struct pci_controller *hose)
{
#ifdef CONFIG_PCI_MSI
    phys_addr_t immr_base;

    immr_base = get_immrbase ();
    early_write_config_dword (hose, 0, 0, PCI_BASE_ADDRESS_0, immr_base);
#endif
}
```

系统软件除了需要设置 PCIe 设备的 Message Address 字段和 PEXCSRBAR 窗口之外，还需要设置 PCIe 设备的 Message Data 字段。PCIe 设备向 MSIIR 寄存器进行存储器写操作的数据存放在 Message Data 字段中。

系统软件在初始化 Message Data 字段之前，首先根据 Multiple Message Capable 字段预先存放的数据初始化 Multiple Message Enable 字段。一个 PCIe 设备最多可以申请 32 个中断请求，但是系统软件根据当前处理器系统的中断资源的使用情况，决定给这个 PCIe 设备提供多少个中断向量，并将这个结果存放到 Multiple Message Enable 字段。

MPC8572 处理器最多可以为 PCIe 设备提供 256 个 MSI 中断请求。但是在某些极端的情况下，可能会出现 PCIe 设备需要的中断请求超过系统所能提供的中断请求。此时某些 PCIe 设备的 Multiple Message Enable 字段可能会小于 Multiple Message Capable 字段。

如果在 PCIe 设备中，使用了多个中断请求，那么 Message Data 字段存放的是一组中断向量号，而 Message Data 字段存放这组中断向量号的基地址。MSI 机制要求“这组数据”连续，其范围在 Message Data ~ Message Data + Multiple Message Enable - 1 之间。在多数情况下，MPC8572 处理器系统仅为一个 PCIe 设备分配 1 个中断向量号。

由上所述，在 MPC8572 处理器系统中，PCIe 设备使用存储器写 TLP 传送 MSI 中断报文，这个存储器写 TLP 使用的地址为 PCIe 设备 Capability 结构的 Message Address 字段，而

Ⓐ 在 Linux PowerPC 中使用 immr_base 变量保存该寄存器。IMMR 寄存器是 PQ2 处理器使用的寄存器，该寄存器在 PQ3 之后的处理器中升级为 CCSRBAR。

Ⓑ 该函数来自 Linux 2.6.30.5 内核。

数据为 Message Data ~ Message Data+Multiple Message Enable-1 之间。其中 Message Data 字段与 MSIIR 寄存器要求的格式相同。

这个特殊的存储器写 TLP 报文通过若干 Switch，并穿越 RC 后，最终将数据写入 MSIIR 寄存器中，并设置 MSIIR 寄存器的 SRS 和 IBS 字段，同时将使能 MSIR0~MSIR7 寄存器的相应位，从而向中断控制器提交中断请求（如果 MSIVPR 寄存器的 MSK 位为 1）。MPIC 中断控制器获得该中断请求后，向处理器系统转发这个中断请求，并由处理器系统执行相应的中断服务例程进行中断处理。MPC8572 处理器也可以处理 PCIe 设备的 MSI-X 中断机制，本节对此不做进一步介绍。

10.3 x86 处理器如何处理 MSI-X 中断请求

PCIe 设备发出 MSI-X 中断请求的方法与发出 MSI 中断请求的方法类似，都是向 Message Address 所在的地址写 Message Data 字段包含的数据。只是 MSI-X 中断机制为了支持更多的中断请求，在 MSI-X Capability 结构中存放了一个指向一组 Message Address 和 Message Data 字段的指针，从而一个 PCIe 设备可以支持的 MSI-X 中断请求数目大于 32 个，而且并不要求中断向量号连续。MSI-X 机制使用的这组 Message Address 和 Message Data 字段存放在 PCIe 设备的 BAR 空间中，而不是在 PCIe 设备的配置空间中，从而可以由用户决定使用 MSI-X 中断请求的数目。

当系统软件初始化 PCIe 设备时，如果该 PCIe 设备使用 MSI-X 机制传递中断请求，需要对 MSI-X Capability 结构指向的 Message Address 和 Message Data 字段进行设置，并使能 MSI-X Enable 位。x86 处理器在此处的实现与 PowerPC 处理器有较大的不同。

10.3.1 Message Address 字段和 Message Data 字段的格式

在 x86 处理器系统中，PCIe 设备也是通过向 Message Address 写入 Message Data 指定的数值实现 MSI/MSI-X 机制。在 x86 处理器系统中，PCIe 设备使用的 Message Address 字段和 Message Data 字段与 PowerPC 处理器不同。

1. PCIe 设备使用 Message Address 字段

在 x86 处理器系统中，PCIe 设备使用的 Message Address 字段仍然保存 PCI 总线域的地址，其格式如图 10-7 所示。



图 10-7 Message Address 字段的格式

其中第 31~20 位存放 FSB Interrupts 存储器空间的基地址，其值为 0xFEE。当 PCIe 设备对 0xFEEX-XXXX 这段“PCI 总线域”的地址空间进行写操作时，MCH/ICH 会首先进行“PCI 总线域”到“存储器域”的地址转换，之后将这个写操作翻译为 FSB 总线的 Interrupt Message 总线事务，从而向 CPU 内核提交中断请求。

x86 处理器使用 FSB Interrupt Message 总线事务转发 MSI/MSI-X 中断请求。使用这种方法的优点是向 CPU 内核提交中断请求的同时，提交 PCIe 设备使用的中断向量，从而 CPU

不需要使用中断响应周期从寄存器中获得中断向量。FSB Interrupt Message 总线事务的详细说明见下文。

Message Address 字段其他位的含义如下所示。

- Destination ID 字段保存目标 CPU 的 ID 号，目标 CPU 的 ID 与该字段相等时，目标 CPU 将接收这个 Interrupt Message。FSB Interrupt Message 总线事务可以向不同的 CPU 提交中断请求。
- RH (Redirection Hint Indication) 位为 0 时，表示 Interrupt Message 将直接发向与 Destination ID 字段相同的目标 CPU；如果 RH 为 1 时，将使能中断转发功能。
- DM (Destination Mode) 位表示在传递优先权最低的中断请求时，Destination ID 字段是否被翻译为 Logical 或者 Physical APIC ID。在 x86 处理器中 APIC ID 有三种模式，分别为 Physical、Logical 和 Cluster ID 模式。
- 如果 RH 位为 1 且 DM 位为 0 时，Destination ID 字段使用 Physical 模式；如果 RH 位为 1 且 DM 位为 1，Destination ID 字段使用 Logical 模式；如果 RH 位为 0，DM 位将被忽略。

以上这些字段的描述与 x86 处理器使用的 APIC 中断控制器相关。对 APIC 的详细说明超出了本书的范围，对此部分感兴趣的读者请参阅 Intel 64 and IA-32 Architectures Software Developer's Manual Volume 3A: System Programming Guide, Part 1。

2. Message Data 字段

Message Data 字段的格式如图 10-8 所示。

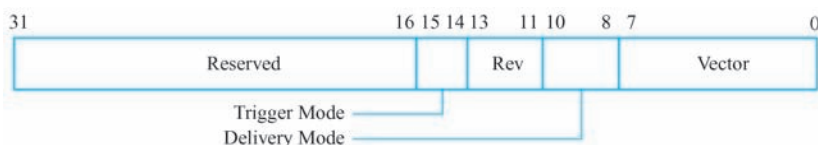


图 10-8 Message Data 字段的格式

Trigger Mode 字段为 0b0x 时，PCIe 设备使用边沿触发方式申请中断；为 0b10 时使用低电平触发方式；为 0b11 时使用高电平触发方式。MSI/MSI-X 中断请求使用边沿触发方式，但是 FSB Interrupt Message 总线事务还支持 Legacy INTx 中断请求方式，因此在 Message Data 字段中仍然支持电平触发方式。但是对于 PCIe 设备而言，该字段为 0b0x。

Vector 字段表示这个中断请求使用的中断向量。FSB Interrupt Message 总线事务在提交中断请求的同时，将中断向量也通知给处理器。因此使用 FSB Interrupt Message 总线事务时，处理器不需要使用中断响应周期通过读取中断控制器获得中断向量号。与 PowerPC 的传统方式相比，x86 处理器的这种中断请求的效率较高^①。

值得注意的是，在 x86 处理器中，MSI 机制使用的 Message Data 字段与 MSI-X 机制相同。但是当一个 PCIe 设备支持多个 MSI 中断请求时，其 Message Data 字段必须是连续的，因而其使用的 Vector 字段也必须是连续的，这也是在 x86 处理器系统中，PCIe 设备支持多个 MSI 中断请求的问题所在，而使用 MSI-X 机制有效避免了该问题。

^① P4080 处理器也提供了一种类似于 FSB Interrupt Message 总线事务的中断请求方法。

Delivery Mode 字段表示如何处理来自 PCIe 设备的中断请求。

- 该字段为 0b000 时，表示使用“Fixed Mode”方式。此时这个中断请求将被 Destination ID 字段指定的 CPU 处理。
- 该字段为 0b001 时，表示使用“Lowest Priority”方式。此时这个中断请求将被优先权最低的 CPU 处理。当使用“Fixed Mode”和“Lowest Priority”方式时，如果 Vector 字段有效，CPU 接收到这个中断请求之后，将使用 Vector 字段指定的中断向量处理这些中断请求；而当 Delivery Mode 字段为其他值时，Message Data 字段中所包含的 Vector 字段无效。
- 该字段为 0b010 时，表示使用 SMI 方式传递中断请求，而且必须使用边沿触发，此时 Vector 字段必须为 0。这个中断请求将被 Destination ID 字段指定的 CPU 处理。
- 该字段为 0b100 时，表示使用 NMI 方式传递中断请求，而且必须使用边沿触发，此时 Vector 字段和 Trigger 字段的内容将被忽略。这个中断请求将被 Destination ID 字段指定的 CPU 处理。
- 该字段为 0b101 时，表示使用 INIT 方式传递中断请求，Vector 字段和 Trigger 字段的内容将被忽略。这个中断请求将被 Destination ID 字段指定的 CPU 处理。
- 该字段为 0b111 时，表示使用 INTR 信号传递中断请求且使用边沿触发。此时 MSI 中断信息首先传递给中断控制器，然后中断控制器通过 INTR 信号向 CPU 传递中断请求，之后 CPU 通过中断响应周期获得中断向量。上文中 PowerPC 处理器使用的方法与此方法类似。而在 x86 处理器中多使用 Interrupt Message 总线事务进行 MSI 中断信息的传递，因此这种模式很少使用。

边沿触发和电平触发是中断请求常用的两种方式。其中电平触发指外部设备使用逻辑电平 1（高电平触发）或者 0（低电平触发），提交中断请求。使用电平或者边沿方式提交中断请求时，外部设备一般通过中断线（IRQ_PIN#）与中断控制器相连，其中多个外部设备可能通过相同的中断线与中断控制器相连（线与或者与门）。

外部设备在使用低电平触发提交中断请求的过程中，首先需要将 IRQ_PIN#信号驱动为低。当中断控制器将该中断请求提交给处理器，而且处理器将这个中断请求处理完毕后，处理器将通过写外部设备的某个寄存器来清除此中断源，此时外部设备将不再驱动 IRQ_PIN#信号线，从而结束整个中断请求。

IRQ_PIN#信号线可以被多个外部设备共享，在这种情况下，只有所有外部设备都不驱动 IRQ_PIN#信号线时，IRQ_PIN#信号才为高电平。采用电平触发方式进行中断请求的优点是不会丢失中断请求，而缺点是一个优先权较高的中断请求有可能会长期占用中断资源，从而使其他优先权较低的中断不能被及时提交。因为优先级别较高的中断源可能会持续不断地驱动 IRQ_PIN#信号。

而边沿触发使用上升沿（0 到 1）或者下降沿（1 到 0）作为触发条件，但是中断控制器并不是使用这个“边沿”作为触发条件。中断控制器使用内部时钟对 IRQ_PIN#信号进行采样，如果在前一个时钟周期，IRQ_PIN#信号为 0，而后一个时钟周期，IRQ_PIN#信号为 1，中断控制器认为外部设备提交了一个有效“上升沿”，中断控制器会锁定这个“上升沿”并向处理器发出中断请求。这也是外部设备至少需要将 IRQ_PIN#信号保持一个时钟采样周期的原因，否则中断控制器可能无法识别本次边沿触发的中断请求，从而产生 Spurious 中断

请求。

外部设备使用“上升沿”进行中断申请时，不需要持续地将 IRQ_PIN#信号驱动为 1，而只需要保证中断控制器可以进行正确采样这些中断信号即可。在处理边沿触发中断请求时，处理器不需要清除中断源。

使用边沿触发可以有效避免“优先级别”较高的中断源长期占用 IRQ_PIN#信号的情况，使用“下降沿”触发进行中断请求与“上升沿”触发类似。

但是外部设备使用边沿触发方式时，有可能会丢失一些中断请求。例如在一个处理器系统中，存在一个定时器，这个定时器使用上升沿触发方式向中断控制器定时提交中断。当处理器正在处理这个定时器的上一个中断请求时，将不会处理这个定时器发出的其他“边沿”中断请求，从而导致中断丢失。而使用电平触发方式不会出现这类问题，因为电平触发方式是一个“持续”过程，处理器只有处理完毕当前中断，并清除相应中断源之后，才会处理下一个中断源。

MSI 中断请求实际上和边沿触发方式非常类似，MSI 中断请求通过存储器写 TLP 实现，这个写动作是一个瞬间的动作，并不是一个持续请求，因此在 x86 处理器中 MSI 中断请求使用边沿触发进行中断请求。

还有一些外部设备可以通过 I/O APIC 进行中断请求^①，这些 I/O APIC 接收的外部中断需要标明是使用边沿或者电平触发，I/O APIC 使用 FSB Interrupt Message 总线事务将中断请求发向 Local APIC，并由 Local APIC 向处理器提交中断请求。

10.3.2 FSB Interrupt Message 总线事务

与 MPC8572 处理器处理 MSI 中断请求不同，x86 处理器使用 FSB 的 Interrupt Message 总线事务，处理 PCIe 设备的 MSI/MSI-X 中断请求。由上文所示，MPC8572 处理器处理 MSI 中断请求时，首先由 MPIC 中断控制器截获这个 MSI 中断请求，之后由 MPIC 中断控制器向 CPU 提交中断请求，而 CPU 通过中断响应周期从 MPIC 中断控制器的 ACK 寄存器中获得中断向量。

采用这种方式的主要问题是，当一个处理器中存在多个 CPU 时，这些 CPU 都需要通过中断响应周期从 MPIC 中断控制器的 ACK 寄存器中获得中断向量。在一个中断较为密集的应用中，ACK 寄存器很可能会成为系统瓶颈。而采用 Interrupt Message 总线事务可以有效地避免这种系统瓶颈，因为使用这种方式中断信息和中断向量将同时到达指定的 CPU，而不需要使用中断响应周期获得中断向量。

x86 处理器也具有通过中断控制器提交 MSI/MSI-X 中断请求的方法，在 I/O APIC 具有一个“The IRQ Pin Assertion Register”寄存器，该寄存器地址为 0xFEC00020^②，其第 4~0 位存放 IRQ Number。系统软件可以将 PCIe 设备的 Message Address 寄存器设置为 0xFEC00020，将 Message Data 寄存器设置为相应的 IRQ Number。

当 PCIe 设备需要提交 MSI 中断请求时，将向 PCI 总线域的 0xFEC00020 地址写入 Interrupt Message Data 寄存器中的数据。此时这个存储器写请求将数据写入 I/O APIC 的 The IRQ Pin

① 与 I/O APIC 的 IRQX#引脚连接的外部设备。

② 该寄存器在存储器域和 PCI 总线域中的地址都为 0xFEC00020。

Assertion Register 中，并由 I/O APIC 将这个 MSI 中断请求最终发向 Local APIC，之后再由 Local APIC 通过 INTR# 信号向 CPU 提交中断请求。

上述步骤与 MPC8572 处理器传递 MSI 中断的方法类似。在 x86 处理器中，这种方式基本上已被弃用。下面以图 10-9 为例，说明 x86 处理器如何使用 FSB 总线的 Interrupt Message 总线事务，向 CPU 提交 MSI/MSI-X 中断请求。

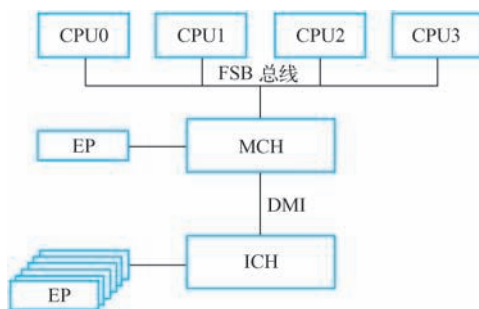


图 10-9 使用 Interrupt Message 总线事务提交 MSI 中断请求

PCIe 设备在发送 MSI/MSI-X 中断请求之前，系统软件需要合理设置 PCIe 设备 MSI/MSI-X Capability 寄存器，使 Message Address 寄存器的值为 $0xFEE_{xx}00y^{\ominus}$ ，同时合理地设置 Message Data 寄存器 Vector 字段。

PCIe 设备提交 MSI/MSI-X 中断请求时，需要向 $0xFEE_{xx}00y$ 地址写 Message Data 寄存器中包含的数据，并以存储器写 TLP 的形式发送到 RC。如果 ICH 收到这个存储器写 TLP 时，将通过 DMI 接口将这个 TLP 提交到 MCH。MCH 收到这个 TLP 后，发现这个 TLP 的目的地址在 FSB Interrupts 存储器空间中，则将 PCIe 总线的存储器写请求转换为 Interrupt Message 总线事务，并在 FSB 总线上广播。

FSB 总线上的 CPU，根据 APIC ID 信息，选择是否接收这个 Interrupt Message 总线事务，并进入中断状态，之后该 CPU 将直接从这个总线事务中获得中断向量号，执行相应的中断服务例程，而不需要从 APIC 中断控制器获得中断向量。与 PowerPC 处理器的 MPIC 中断控制器相比，这种方法更具优势。

10.4 小结

本章详细描述了 MSI/MSI-X 中断机制的原理，并以 PowerPC 和 x86 两个处理器系统为例说明这两种中断机制实现机制。本章因为篇幅有限，并没有详细讲述这两个处理器使用的中断控制器。而理解这些中断控制器的实现机制是进一步理解 MSI/MSI-X 中断机制的要点。对此部分有兴趣的读者可以继续阅读 MPIC 中断控制器和 APIC 中断控制器的实现机制，以加深对 MSI/MSI-X 中断机制的理解。

设备的中断处理是局部总线的设计难点和重要组成部分，而中断处理的效率直接决定了局部总线的数据传送效率。在一个处理器系统的设计与实现中，中断处理的优化贯彻始终。

[⊖] 其中 xx 表示 APIC ID，而 y 为 RH+DM。